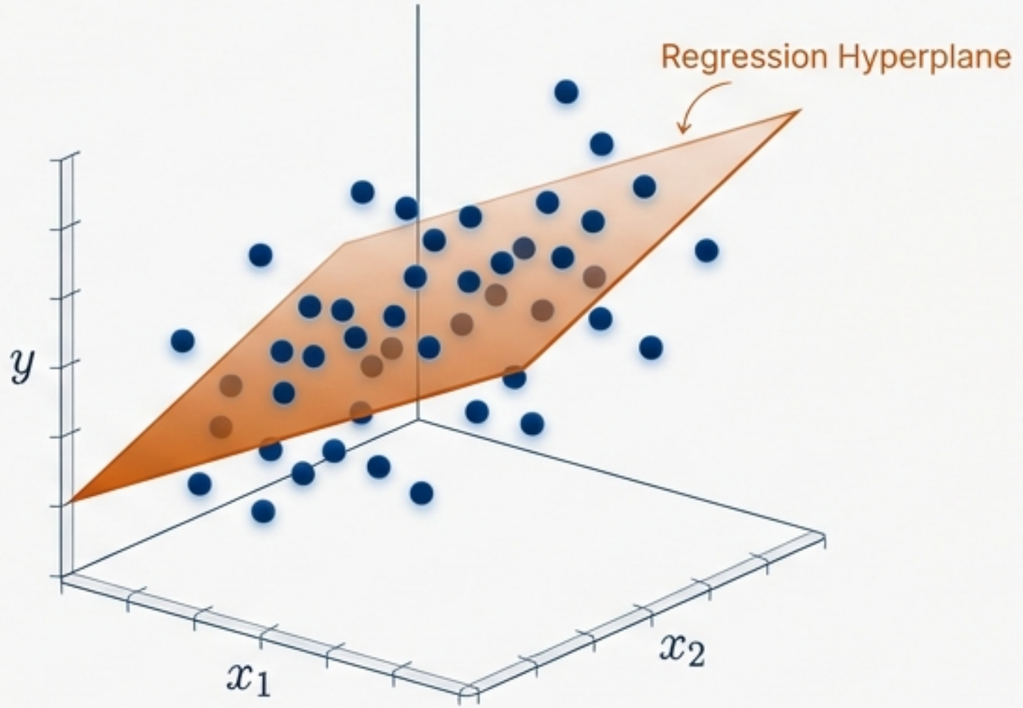
Multiple Linear Regression: Mathematical Formulation From Scratch

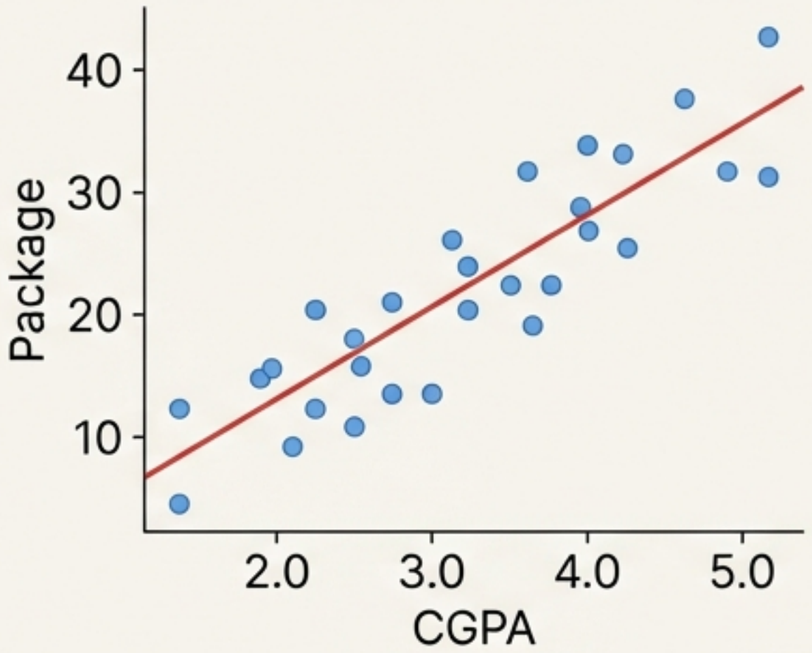
Deriving the Normal Equation using Ordinary Least Squares (OLS)



$$\beta = (X^T X)^{-1} X^T Y$$

From 2D Lines to Hyperplanes

Simple Linear Regression

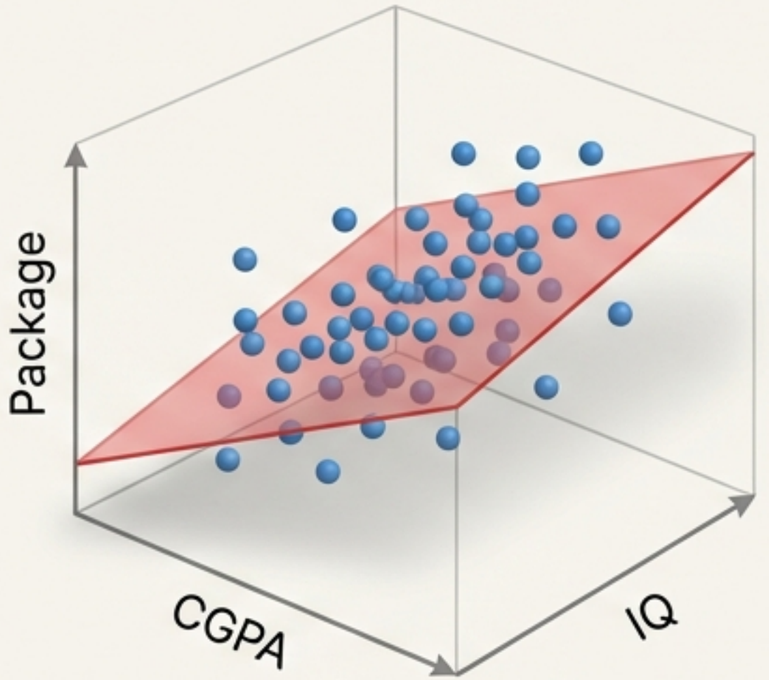


1 Input → 1 Output

Geometry: Line

$$y = \beta_0 + \beta_1 x_1$$

Multiple Linear Regression



Multiple Inputs → 1 Output

Geometry: Hyperplane

$$y = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \ldots + \beta_n x_n$$

Generalising to n features

The Matrix Translation

Converting scalar algebra into linear algebra

Scalar World

CGPA (x_1)	IQ (x_2)	Target (y)
3.2	115	25
3.8	120	32
2.9	110	22
3.5	118	29
...	...	...
... 3.6	... 123	... 20

$$y_i = \beta_0 + \beta_1 x_{i1} + \beta_2 x_{i2}$$

The Bias Trick (Adding $x_0 = 1$)

Matrix World

$$Y = X\beta$$

$$\begin{bmatrix} 1 & 3.2 & 115 \\ 1 & 3.8 & 120 \\ 1 & 2.9 & 110 \\ 1 & 3.5 & 118 \\ 1 & 2.5 & 120 \\ 1 & 3.6 & 125 \\ 1 & 3.7 & 128 \end{bmatrix}$$

Intercept Term (x_0)

Anatomy of the Core Matrices

Understanding the dimensions is key to the derivation

$$Y = \begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_m \end{bmatrix} \qquad \begin{bmatrix} 1 & x_{11} & x_{12} & \cdots & x_{1n} \\ 1 & x_{21} & x_{22} & \cdots & x_{2n} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & x_{m1} & x_{m2} & \cdots & x_{mn} \end{bmatrix} \qquad \beta = \begin{bmatrix} \beta_0 \\ \beta_1 \\ \vdots \\ \beta_n \end{bmatrix}$$

Target Vector
Shape: $m \times 1$
(Actual Values)

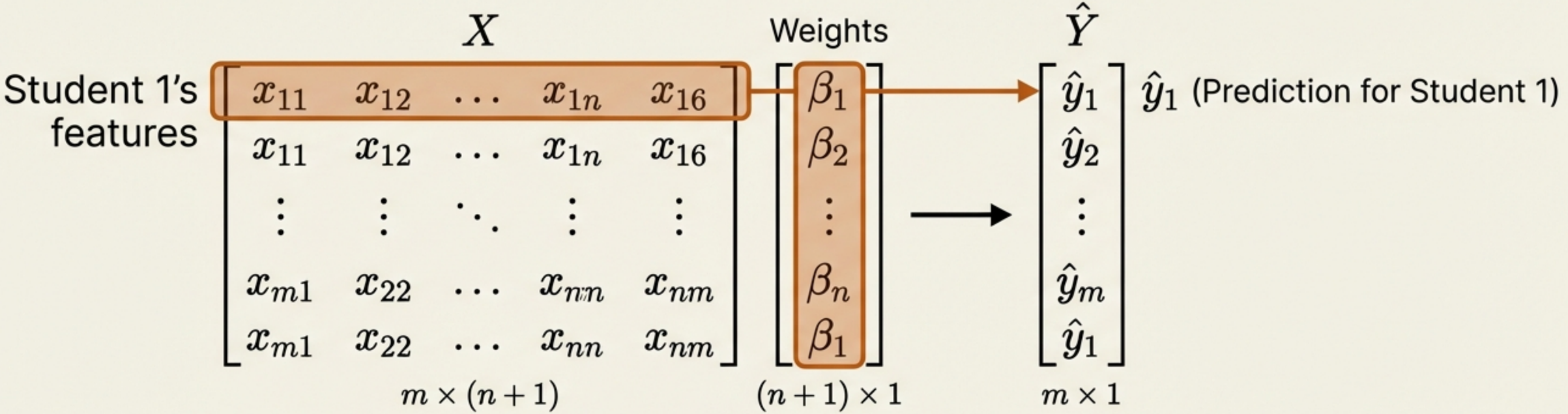
Design Matrix
Shape: $m \times (n+1)$
(Features + Bias)

Coefficient Vector
Shape: $(n+1) \times 1$
(Unknown Weights)

The Prediction Vector ($\hat{Y}$)

Predicting for the entire dataset simultaneously

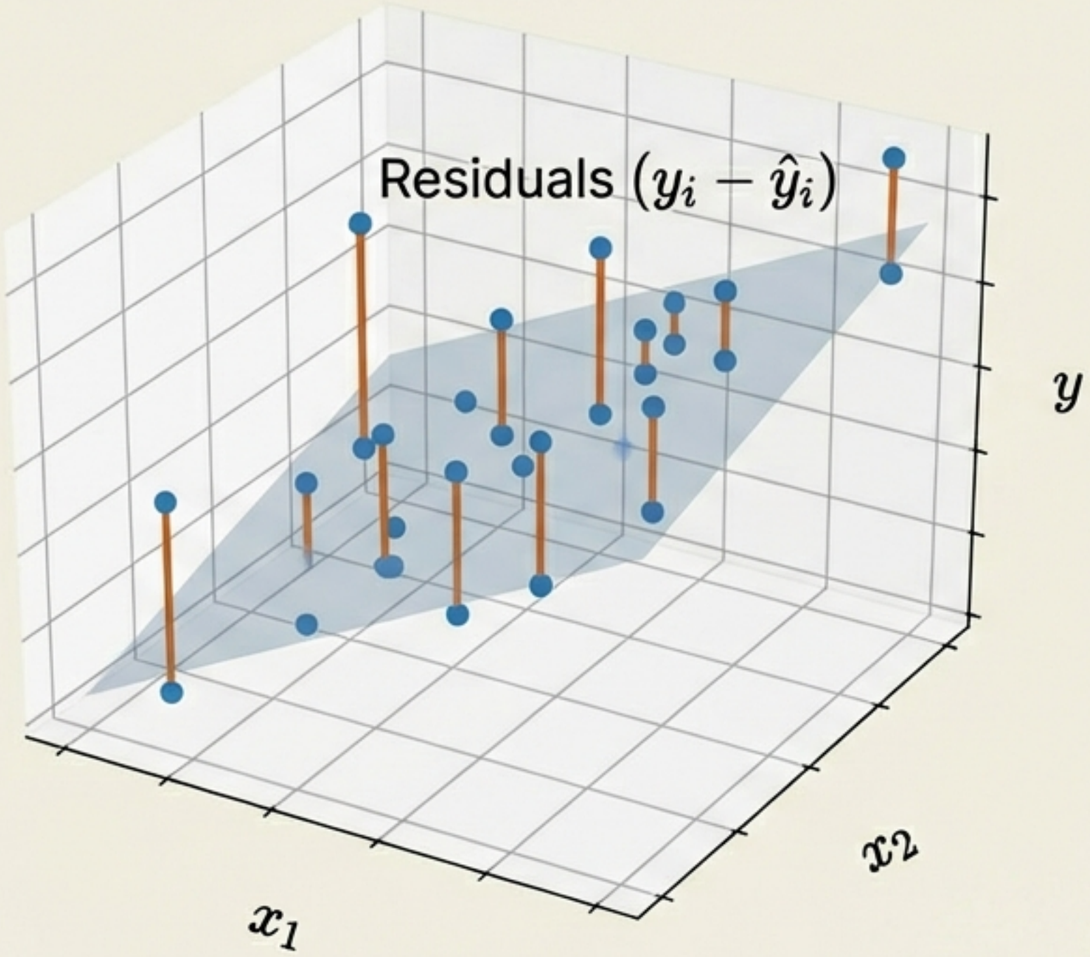
$$\hat{Y} = X\beta$$



Instead of looping through m students, Linear Algebra computes all predictions in a single operation.

Defining the Loss Function

Minimising the Sum of Squared Errors (SSE)



Objective: Minimise the total squared length of these lines.

$$E = \sum_{i=1}^{m} (y_i - \hat{y}_i)^2$$

- y_i: Actual Value
- $\hat{y}_i$: Predicted Value ($x_i\beta$)
- **Square**: To penalise large errors and handle negative values.

Matrix Formulation of Error

Converting summation to vector operations

Define the Error Vector, e:

$$\mathbf{e} = (Y - \hat{Y}) = \begin{bmatrix} y_1 - \hat{y}_1 \\ y_2 - \hat{y}_2 \\ \vdots \\ y_m - \hat{y}_m \end{bmatrix}$$

← Error per instance

The dot product of a vector with itself sums the squares:

$$\mathbf{e}^T\mathbf{e} = \begin{bmatrix} e_1 & e_2 & \cdots & e_m \end{bmatrix} \begin{bmatrix} e_1 \\ e_2 \\ \vdots \\ e_m \end{bmatrix} = \sum e_i^2$$

The Loss Function in Matrix Notation:

$$E = (Y - \hat{Y})^T (Y - \hat{Y})$$

Expanding the Loss Equation

Step-by-step algebraic expansion

Step 1: Substitute $\hat{Y} = X\beta$:

$$E = (Y - X\beta)^T (Y - X\beta)$$

Step 2: Distribute the Transpose $(A-B)^T = A^T - B^T$:

$$E = (Y^T - (X\beta)^T)(Y - X\beta)$$

Step 3: Apply Product Transpose Rule $(AB)^T = B^T A^T$:

$$E = (Y^T - \beta^T X^T)(Y - X\beta)$$

Step 4: Expand (FOIL Method):

$$E = \underset{①}{Y^T Y} - \underset{②}{Y^T X\beta} - \underset{③}{\beta^T X^T Y} + \underset{④}{\beta^T X^T X\beta}$$

Simplifying the Scalar Terms

Exploiting symmetry to combine terms

Exploiting symmetry to combine terms

$$E = \mathbf{Y}^T\mathbf{Y} - \underbrace{\mathbf{Y}^T\mathbf{X}\boldsymbol{\beta}}_{\text{Term A}} - \underbrace{\boldsymbol{\beta}^T\mathbf{X}^T\mathbf{Y}}_{\text{Term B}} + \boldsymbol{\beta}^T\mathbf{X}^T\mathbf{X}\boldsymbol{\beta}$$

Key Insight:
The result of $\mathbf{Y}^T\mathbf{X}\boldsymbol{\beta}$ is a scalar (1 x 1 matrix). The transpose of a scalar is itself.

Check: $(\text{Term B})^T = (\boldsymbol{\beta}^T\mathbf{X}^T\mathbf{Y})^T = \mathbf{Y}^T(\mathbf{X}^T)^T(\boldsymbol{\beta}^T)^T = \mathbf{Y}^T\mathbf{X}\boldsymbol{\beta} = \text{Term A}$

Since Term A = Term B, we can combine them:

$$E = \mathbf{Y}^T\mathbf{Y} - 2\mathbf{Y}^T\mathbf{X}\boldsymbol{\beta} + \boldsymbol{\beta}^T\mathbf{X}^T\mathbf{X}\boldsymbol{\beta}$$

The Calculus: Minimising Loss

Taking the derivative with respect to β

$$\frac{\partial E}{\partial \beta} = \frac{\partial}{\partial \beta}(\mathbf{Y}^T\mathbf{Y} - 2\mathbf{Y}^T\mathbf{X}\beta + \beta^T\mathbf{X}^T\mathbf{X}\beta)$$

$\mathbf{Y}^T\mathbf{Y}$	$-2\mathbf{Y}^T\mathbf{X}\beta$	$\beta^T\mathbf{X}^T\mathbf{X}\beta$
Contains no β	Linear Term (like $2ax$)	Quadratic Term (like ax^2)
Derivative → 0	Derivative → $-2X^TY$	Derivative → $2X^TX\beta$

Setting the gradient to zero:

$$-2X^TY + 2X^TX\beta = 0$$

The Solution: The Normal Equation

$$-2\boldsymbol{X}^T\boldsymbol{Y} + 2\boldsymbol{X}^T\boldsymbol{X}\boldsymbol{\beta} = 0$$

↓ Move negative term to right:

$$2\boldsymbol{X}^T\boldsymbol{X}\boldsymbol{\beta} = 2\boldsymbol{X}^T\boldsymbol{Y}$$

↓ Divide by 2:

$$\boldsymbol{X}^T\boldsymbol{X}\boldsymbol{\beta} = \boldsymbol{X}^T\boldsymbol{Y}$$

↓ Multiply both sides by $(\boldsymbol{X}^T\boldsymbol{X})^{-1}$

$$\boxed{\boldsymbol{\beta} = (\boldsymbol{X}^T\boldsymbol{X})^{-1}\boldsymbol{X}^T\boldsymbol{Y}}$$

The Closed-Form Solution for Multiple Linear Regression

Dimensional Sanity Check

Verifying the matrix shapes match the result

$$\boxed{X^T X}$$

Shape:
$(n+1 \times m) \cdot (m \times n+1)$
$\rightarrow (n+1 \times n+1)$

$$\boxed{(X^T X)^{-1}}$$

Inverse keeps shape:
$(n+1 \times n+1)$

$$\boxed{X^T Y}$$

Shape:
$(n+1 \times m) \cdot (m \times 1)$
$\rightarrow (n+1 \times 1)$

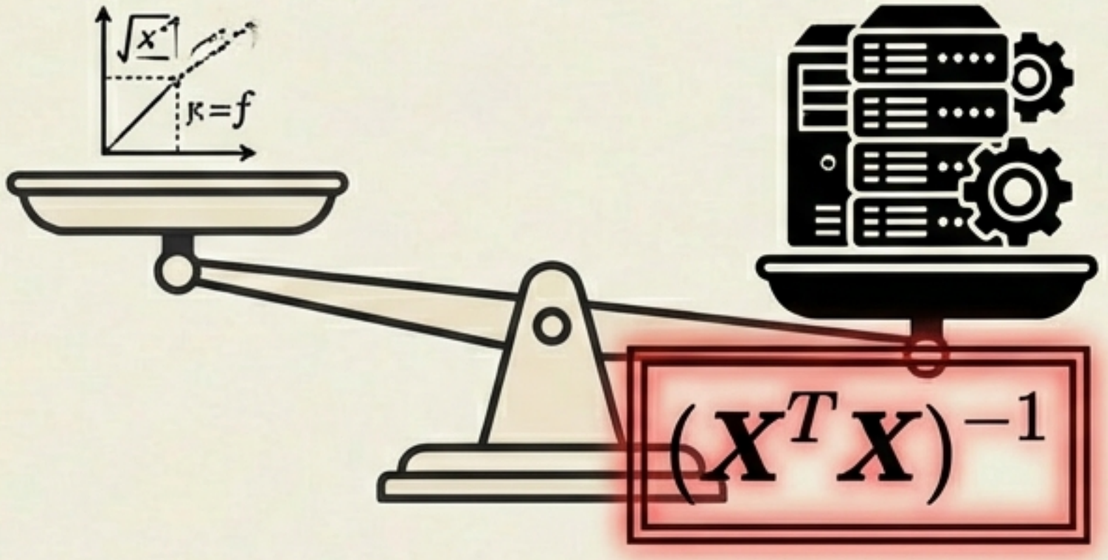
$$(n+1 \times n+1) \cdot (n+1 \times 1)$$

Result β Shape: $(n+1 \times 1)$ ✓

The Computational “Catch”

Why OLS isn’t always the answer

Mathematics
(Elegant, Exact)



Computation
(Expensive)

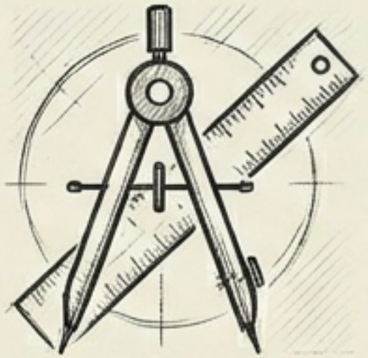
- ****Matrix Inversion Cost****: $O(n^3)$ (Cubic Complexity)
- If features (n) = 10 → 1,000 operations (Instant)
- If features (n) = 10,000 → 1,000,000,000,000 operations (Slow/Impossible)

Strictly limited to datasets with manageable feature counts.

Two Paths to the Same Goal

Choosing the right algorithm for the data

OLS (Normal Equation)



- **Method:** Exact Matrix Algebra
- **Pros:** Deterministic, No hyperparameters
- **Cons:** $O(n^3)$ complexity, slow for large n
- **Scikit-Learn:** `LinearRegression`

Gradient Descent

- **Method:** Iterative Approximation
- **Pros:** Scalable, Efficient for big data
- **Cons:** Approximate, requires Learning Rate tuning
- **Scikit-Learn:** `SGDRegressor`

The Derivation Journey

From intuition to formulation to solution

Formulation

Translate data to Matrix Notation

$$E = (Y - X\beta)^T(Y - X\beta)$$

Minimisation

Apply Calculus to find zero gradient

$$\frac{\partial E}{\partial \beta} = 0$$

Solution

Solve for Coefficients

$$\beta = (X^T X)^{-1} X^T Y$$

We have successfully demystified the ‘Black Box’ of Linear Regression.